

■ Spring Security + JWT Interview Questions (50)

1. What is Spring Security?

Spring Security is a powerful security framework for Java applications.

It provides authentication and authorization mechanisms.

It protects REST APIs and web applications using filters and role-based access.

2. What is Authentication?

Authentication is the process of verifying user identity.

It checks credentials like username and password.

If credentials are valid, the user is considered authenticated.

3. What is Authorization?

Authorization determines what an authenticated user can access.

It works based on roles and authorities.

Access is granted or denied depending on permissions.

4. Difference between Authentication and Authorization?

Authentication verifies identity of the user.

Authorization decides access level of that user.

Authentication happens before authorization.

5. What is SecurityFilterChain?

SecurityFilterChain defines security rules in Spring Boot.

It configures authentication and authorization policies.

It replaces WebSecurityConfigurerAdapter in Spring Security 6.

6. What is a Filter in Spring Security?

Filters intercept incoming HTTP requests.

They validate tokens or credentials before controller execution.

JWT validation is usually done inside a custom filter.

7. What is UserDetailsService?

UserDetailsService loads user-specific data from database.

It returns a UserDetails object to Spring Security.

It is used during authentication process.

8. What is UserDetails?

UserDetails is an interface representing authenticated user data.

It contains username, password, and authorities.

Spring Security uses it internally for validation.

9. What is PasswordEncoder?

PasswordEncoder is used to encode passwords securely.

It prevents storing plain text passwords.

BCryptPasswordEncoder is most commonly used.

10. Why is BCrypt preferred?

BCrypt automatically applies salting.

It is computationally expensive, reducing brute-force attacks.

It increases overall password security.

11. What is JWT?

JWT stands for JSON Web Token.

It is a stateless authentication mechanism.

It securely transfers user information between client and server.

12. Structure of JWT?

JWT consists of Header, Payload, and Signature.

Header defines algorithm and token type.

Signature ensures token integrity and authenticity.

13. What is Stateless Authentication?

Stateless authentication means server does not store session data.

Each request contains authentication information.

JWT is commonly used for stateless systems.

14. Why JWT over Session?

JWT removes need for server-side session storage.

It scales better in distributed systems.

It reduces database load for session validation.

15. What is AuthenticationManager?

AuthenticationManager processes authentication requests.

It delegates validation to AuthenticationProvider.

It returns authenticated object if successful.

16. What is AuthenticationProvider?

AuthenticationProvider contains authentication logic.

It validates credentials against database or external system.

Multiple providers can be configured.

17. What is OncePerRequestFilter?

It ensures a filter runs only once per request.

Used commonly for JWT validation.

It prevents duplicate filter execution.

18. How is JWT validated?

The token signature is verified using secret key.

Expiration time is checked.

Claims are extracted for user identity.

19. What is CSRF?

CSRF stands for Cross-Site Request Forgery.

It tricks users into performing unwanted actions.

For stateless APIs, CSRF is usually disabled.

20. Why disable CSRF in REST APIs?

REST APIs are typically stateless.

They do not use cookies for authentication.

JWT-based systems do not require CSRF protection.

21. What is Role vs Authority?

Role is a high-level permission like `ROLE_ADMIN`.

Authority is a fine-grained permission.

Spring internally converts roles to authorities.

22. What is `@PreAuthorize`?

`@PreAuthorize` is method-level security annotation.

It checks authorization before method execution.

It supports SpEL expressions.

23. What is `@EnableMethodSecurity`?

It enables method-level security.

It allows usage of annotations like `@PreAuthorize`.

It is required in Spring Security 6.

24. What is Bearer Token?

Bearer token is sent in Authorization header.

Format is "Bearer ".

It represents authenticated user identity.

25. Where is JWT stored?

JWT is usually stored in client-side storage.

Common places are `localStorage` or cookies.

It is sent with every request.

26. What are Claims in JWT?

Claims contain user data like username and roles.

They are stored inside payload section.

Claims help identify authenticated user.

27. What is Token Expiration?

JWT has an expiration time defined in payload.

After expiry, token becomes invalid.

User must re-authenticate.

28. What is Refresh Token?

Refresh token is used to generate new access token.

It has longer expiration time.

It improves security by limiting access token validity.

29. How to handle JWT expiration?

Check expiration during validation.

Return 401 Unauthorized if expired.

Client should use refresh token.

30. What is SecurityContext?

SecurityContext holds authentication details.

It stores authenticated user information.

It is available throughout request lifecycle.

31. What is SecurityContextHolder?

SecurityContextHolder stores SecurityContext.

It provides access to current authenticated user.

Used to get logged-in username.

32. What is AccessDeniedException?

It occurs when user lacks required authority.

It results in HTTP 403 Forbidden.

Authentication is successful but authorization fails.

33. What is HttpSecurity?

HttpSecurity configures web-based security.

It defines authorization rules and filters.

It is configured inside SecurityFilterChain.

34. What is formLogin()?

formLogin enables form-based authentication.

It provides default login page.

Mostly used in web applications.

35. What is Basic Authentication?

Basic authentication sends credentials in header.

Credentials are base64 encoded.

It is less secure without HTTPS.

36. How to secure REST API with JWT?

Disable session management.

Add custom JWT filter.

Validate token before processing request.

37. What is OAuth2?

OAuth2 is authorization framework.

It allows third-party login like Google.

Spring Security provides OAuth2 support.

38. What is Resource Server?

Resource server validates JWT tokens.

It protects API endpoints.

It verifies token signature and claims.

39. What is Keycloak?

Keycloak is identity and access management tool.

It provides authentication server.

It integrates with Spring Security.

40. What is Token Tampering?

Token tampering modifies payload.

Signature verification prevents this.

Invalid signature rejects request.

41. What is CORS?

CORS controls cross-origin requests.

It prevents unauthorized domain access.

Must be configured in security config.

42. What is Session Management?

It manages user sessions.

In JWT systems, sessions are disabled.

State is maintained on client side.

43. What is Anonymous Authentication?

Allows access without login.

Anonymous user gets limited permissions.

Useful for public endpoints.

44. What is Custom Authentication Filter?

It is user-defined filter.

Used to validate JWT token.

Extends `OncePerRequestFilter`.

45. What is `AuthenticationEntryPoint`?

Handles unauthorized access attempts.

Returns 401 Unauthorized response.

Triggered when authentication fails.

46. What is Logout Handling?

Spring Security provides logout endpoint.

It clears `SecurityContext`.

JWT logout usually handled client-side.

47. What is Method-Level Security?

Security applied at method level.

Uses annotations like `@PreAuthorize`.

Provides fine-grained access control.

48. How to get logged-in user?

Use `SecurityContextHolder`.

Call `getAuthentication().getName()`.

It returns current username.

49. What is `ExceptionHandler` in Security?

Handles security-related exceptions.

Customizes 401 and 403 responses.

Improves API error handling.

50. How JWT improves Microservices security?

It enables stateless authentication.

No centralized session storage required.

Each service can validate token independently.